U3lib Manual (For Internal Use Only)

Phong Dang[1, 2]

[1] *Department of Physics and Astronomy, Louisiana State University, Baton Rouge, LA, USA*
[2] *Quantum CodeX, Baton Rouge, LA, USA*

Abstract: A manual of the C++ library U3lib for fast and accurate evaluation of U(3) Wigner coupling and Racah recoupling coefficients is presented. It is implemented based on a new mathematical procedure introduced by Dang et al. [1], which avoids the utilization of binomial coefficients and alternating sums that have been used in the 1973 formulation of Draayer and Akiyama. The library also features a program that evaluates reduced matrix elements of SU(3) physical generators $\mathbf{L}_\mu$ and $\mathbf{Q}_\mu$, which can be used to calculate matrix elements of operators that are powers and products of those generators.

Keywords: U(3); SU(3); SO(3); Wigner coefficients; Racah coefficients; Clebsch-Gordan coefficients; Nuclear structure

PROGRAM SUMMARY

Program Title: U3lib
Programming Language: C++
External library: Eigen3 [2], WIGXJPF [3]
Nature of problem: Fast and accurate evaluation of Wigner coupling coefficients for two group-subgroup chains, namely SU(3) ⊃ SO(3) and U(3) ⊃ U(2) ⊃ U(1), and Racah recoupling coefficients for U(3).

---

[1] P. Dang, J.P. Draayer, F. Pan and K.S. Becker;"New Procedure for Evaluation of U(3) Coupling and Recoupling Coefficients", Eur. Phys. J. Plus **139**, 704 (2024)
[2] Gaël Guennebaud, Benoît Jacob, et al.; `http://eigen.tuxfamily.org` (2010)
[3] H.T. Johansson and C. Forssén; SIAM J. Sci. Comput. **38**(1), A376 (2016)

I. INTRODUCTION

A. Data structures for irreps and basis states

All data structures given in this subsection are defined in `src/U3basis.h`.

1. U(3) irreps are stored using a structure called `u3::irrep`, whose members are `n13`, `n23`, `n33`, corresponding to the three quantum numbers of a U(3) irrep. The default constructor sets all three quantum numbers to zero. When the quantum numbers are known, there are two ways to initialize an irrep

- `irrep(n1,n2,n3)`: the three quantum numbers are set to the given values `n1`, `n2`, and `n3`, respectively;
- `irrep(lambda,mu)`: initialize a U(3) irrep from Elliott's SU(3) labels.

There are three important methods

- `dim()`: return the dimension of a U(3) irrep $[n_{13}, n_{23}, n_{33}]$ as an integer;
- `check()`: a boolean-type method to check whether the quantum numbers are valid for an irrep, i.e., $n_{13} \geq n_{23} \geq n_{33}$. This method is particularly helpful in finding the decomposition of the tensor product of two U(3) irreps;
- `normalize()`: change a U(3) irrep $[n_{13}, n_{23}, n_{33}]$ to $[n_{13} - n_{33}, n_{23} - n_{33}, 0]$. This is important because U(3) irreps $[n_{13}, n_{23}, n_{33}]$ and $[n_{13} + m, n_{23} + m, n_{33} + m]$ (where m is an integer) are equivalent, thus we need to avoid store coupling coefficients associated with equivalent irreps.

2. Gelfand-Tsetlin patterns – also called Gelfand states for short:

$$|G\rangle = \left| \begin{matrix} n_{13}, n_{23}, n_{33} \\ n_{12}, n_{22} \\ n_{11} \end{matrix} \right\rangle, \text{ where the } \textit{betweenness} \text{ condition must be satisfied: } n_{ij} \geq n_{i,j-1} \geq n_{i+1,j},$$

are stored using a structure called `u3::gelfand`, which is composed of six elements `n13`, `n23`, `n33`, `n12`, `n22`, `n11`. The default constructor sets all elements to zero, whereas if all six quantum numbers are known, a Gelfand state can be initialized by passing the values of the quantum numbers in their order. A boolean-type method called `check()` serves to ensure that the betweenness condition is satisfied. There are 13 other methods, namely `e11()`, `e22()`, `e33()`, `e12()`, `e21()`, `e23_12()`, `e23_22()`, `e32_12()`, `e32_22()`, `e13_12()`, `e13_22()`, `e31_12()`, `e31_22()`, which compute the coefficients that result from the action of the 9 generators of U(3), see Eqs. (5)-(7) in [1].

An important method is `normalize()`, which normalizes a Gelfand state

$$\left| \begin{matrix} n_{13}, n_{23}, n_{33} \\ n_{12}, n_{22} \\ n_{11} \end{matrix} \right\rangle \longrightarrow \left| \begin{matrix} n_{13} - n_{33}, n_{23} - n_{33}, 0 \\ n_{12} - n_{33}, n_{22} - n_{33} \\ n_{11} - n_{33} \end{matrix} \right\rangle.$$

The purpose of this method is the same as that of `normalize()` in `u3::irrep`.

3. Another data structure called `u3::physicalvector` is used to store physical basis states

$$\left| \begin{matrix} n_{13}, n_{23}, 0 \\ \kappa L M = L \end{matrix} \right\rangle,$$

which are the highest weight state of the physical reduction SU(3) $\supset$ SO(3) with κ or `kappa` (in the program) is a counter for multiple occurrences of L in a U(3) irrep.

B. Multiplicity convention

The inner multiplicity `kappa` that occurs in the reduction to SO(3) and the outer multiplicity `rho` occurring in the decomposition of a tensor product of two U(3) irreps into a direct sum of U(3) irreps are displayed starting at 1; i.e.,

- `kappa = 1, 2, ..., Kappa(u3::irrep,l)` where `l` is the angular momentum quantum number;
- `rho = 1, 2, ..., rhomax`.

Location	Function name	Task/Computation	References
`src/decomposition.h`	`decomposition`	decomposition of the tensor product $[n_{13}, n_{23}, n_{33}] \otimes [n'_{13}, n'_{23}, n'_{33}] = \oplus\rho_{\max}[n''_{13}, n''_{23}, n''_{33}]$	Sec. IV.A in [1]
`src/U3U2U1xU3U2U1.h`	`HighestWeightCoeffs`	U(3) ⊃ U(2) ⊃ U(1) coupling coefficients for the highest weight state of the coupled irrep	Sec. IV.C in [1]
	`LowerWeightCoeffs`	U(3) ⊃ U(2) ⊃ U(1) coupling coefficients for the lower weight states of the coupled irrep	Sec. IV.D in [1]
`src/U3_U6Z6Coeffs.h`	`U6`	6-U(3) Racah recoupling U-coefficients	Eq. (21) in [1]
	`Z6`	6-U(3) Racah recoupling Z-coefficients	Eq. (22) in [1]
`src/U3_U9Coeffs.h`	`U9`	9-U(3) Racah recoupling coefficients	Eq. (23) in [1]
`src/U3SO3_to_U3U2U1.h`	`phys_to_cano`	expansion of the highest weight SU(3) ⊃ SO(3) ⊃ SO(2) basis state ($M = L$) in terms of Gelfand states	Eqs. (35),(36) in [1]
`src/U3SO3xU3SO3.h`	`WigU3SO3`	SU(3) ⊃ SO(3) Wigner coefficients	Eq. (40) in [1]
`src/U3generators.h`	`RME_L`	reduced matrix elements $\langle(\lambda\mu)\kappa' L\|\|\mathbf{L}\|\|(\lambda\mu)\kappa L\rangle$ for all multiplicities	Eq. (7) in [2]
	`RME_Q`	reduced matrix elements $\langle(\lambda\mu)\kappa' L'\|\|\mathbf{Q}\|\|(\lambda\mu)\kappa L\rangle$ for all multiplicities	Eqs. (51)-(53) in [3]

TABLE I: Main functions of the U3lib library.

II. LIBRARY INTERFACE

The U3lib library provides simple programs for the evaluation of U(3) Wigner coupling and recoupling coefficients and a program for computing reduced matrix elements of SU(3) physical generators. The main directory contains `.cpp` programs that serve as example for calling functions and printing outputs, whereas the implementation can be found in the header files of the same names in the `src` directory.

It is worth noting that all normalizations are done in the .cpp programs when finalizing and printing out the outputs. Outputs from functions in header files are not normalized and I highly suggest not messing with the header files (except the utility files) because it may affect the computation.

Table I lists all the header files and the main functions (contained in namespace `u3`) therein together with their tasks and a reference to the mathematical detail. In what follows, details of these functions – regarding inputs and outputs – will be given.

A. Calculation of U(3) coupling coefficients

In this subsection, details in terms of inputs and outputs of each main function in the namespace `u3` shown in Table I are given.

1. Decomposition:

```
std::vector<u3::irrep> decomposition(u3::irrep &u31, u3::irrep &u32,
                                     std::vector<int> &rhomaxs)
```

Admits two U(3) irreps as input, and returns an array of U(3) coupled irreps with the outer multiplicities being stored in the array `rhomaxs`.

2. U(3) ⊃ U(2) ⊃ U(1) coupling coefficients of the highest weight state:

```
void HighestWeightCoeffs(u3::irrep &u3a, u3::irrep &u3b, u3::gelfand &HWc,
                         std::vector<u3::gelfand> &gArr1, std::vector<u3::gelfand> &gArr2,
                         std::vector<u3::gelfand> &gArr3, Eigen::Matrix<double,Eigen::Dynamic,
                         Eigen::Dynamic> &coeffs)
```

Takes two U(3) irreps and the highest weight state of the coupled irrep as inputs. Since the coupling coefficients have three entries, $C^{HW''\rho}_{G,G'}$, each of them is stored separately, i.e., $|G\rangle$ and $|G'\rangle$ are stored in the two arrays `gArr1` and `gArr2`, whereas `gArr3` simply contains multiple copies of $|HW''\rangle$. The coupling coefficients are stored in matrix `coeffs`, which has `rhomax` columns.

3. U(3) ⊃ U(2) ⊃ U(1) coupling coefficients of lower weight states of the same p-weight:

```
void LowerWeightCoeffs(u3::irrep &u3a, u3::irrep &u3b, u3::irrep &u3c, int w3, int w1,
                       std::vector<u3::gelfand> &gArr1,
                       std::vector<u3::gelfand> &gArr2,
                       std::vector<u3::gelfand> &gArr3,
                       Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> &coeffs)
```

Takes three U(3) irreps and the p-weight that is of interest as inputs. Similarly to `HeighestWeightCoeffs`, it returns three entries of the CGC's, namely, $|G\rangle, |G'\rangle, |G''\rangle$, into three arrays called `gArr1`, `gArr2`, `gArr3`, and the coefficients themselves are stored in a matrix whose number of columns is equal to `rhomax`.

4. Transformation between SU(3) ⊃ SO(3) and U(3) ⊃ U(2) ⊃ U(1) bases:

```
Eigen::Matrix<double, Eigen::Dynamic,Eigen::Dynamic> phys_to_cano(physicalvector &physvec,
                              Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> &Lproj)
```

Admits a highest weight state ($M = L$) and expands it in the canonical basis. All the expansion coefficients are returned in a matrix whose number of columns is equal to the inner multiplicity $\kappa_{\max}$, that occurs in the reduction to SO(3).

5. Wigner SU(3) ⊃ SO(3) coupling coefficients:

```
Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> WigU3SO3(u3::irrep &u3A, u3::irrep &u3B,
                              u3::irrep &u3C, std::vector<u3::physicalvector> &physvecA,
                              std::vector<u3::physicalvector> &physvecB,
                              std::vector<u3::physicalvector> &physvecC)
```

Accepts three irreps as input and generates all coupling coefficients which are stored in a matrix with `rhomax` columns. The three entries of the coefficients

$$\left| \begin{matrix} n_{13}, n_{23}, 0 \\ \kappa LM = L \end{matrix} \right\rangle, \left| \begin{matrix} n'_{13}, n'_{23}, 0 \\ \kappa' L' M' = L' \end{matrix} \right\rangle, \text{ and } \left| \begin{matrix} n''_{13}, n''_{23}, 0 \\ \kappa'' L'' M'' = L'' \end{matrix} \right\rangle,$$

are stored in three arrays, namely `physvecA`, `physvecB`, and `physvecC`.

B. Calculation of U(3) recoupling coefficients

1. 6-U(3) Racah recoupling U-coefficients:

```
std::vector<double> U6(u3::irrep &u3a, u3::irrep &u3b, u3::irrep &u3abc, u3::irrep &u3c,
                       u3::irrep &u3ab, std::vector<int> &rhoab, std::vector<int> &rhoab_c,
                       u3::irrep &u3bc, std::vector<int> &rhobc, std::vector<int> &rhoa_bc)
```

Admits six irreps, three of which are the three irreps that are coupled to a final irrep, whereas two irreps are the resultants of coupling the first and the second, and of coupling the second and the third. The function returns all recoupling coefficients for all outer multiplicities, stored in an array `std::vector<double>`.

2. 6-U(3) Racah recoupling Z-coefficients:

```
std::vector<double> Z6(u3::irrep &u3b, u3::irrep &u3a, u3::irrep &u3abc, u3::irrep &u3c,
                       u3::irrep &u3ab, std::vector<int> &rhoab, std::vector<int> &rhoab_c,
                       u3::irrep &u3ac, std::vector<int> &rhoac, std::vector<int> &rhoac_b)
```

Admits six irreps, three of which are the three irreps that are coupled to a final irrep, whereas two irreps are the resultants of coupling the first and the second, and of coupling the first and the third. The function returns all recoupling coefficients for all outer multiplicities, stored in `std::vector<double>`.

3. 9-U(3) Racah recoupling coefficients:

```
Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> U9(u3::irrep &u3a, u3::irrep &u3b,
                       u3::irrep &u3c, u3::irrep &u3d, u3::irrep &u3ab, u3::irrep &u3cd,
                       u3::irrep &u3ac, u3::irrep &u3bd, u3::irrep &u3abcd, std::vector<int>
                       &rhoab, int rhoab_max, std::vector<int> &rhocd, int rhocd_max,
                       std::vector<int> &rhoab_cd, int rhoab_cd_max, std::vector<int> &rhoac,
                       int rhoac_max, std::vector<int> &rhobd, int rhobd_max, std::vector<int>
                       &rhoac_bd, int rhoac_bd_max)
```

Needs nine inputs, which are the four irreps that are coupled to obtain the final one, the other four are the resultants of coupling the 1st and the 2nd, the 3rd and the 4th, the 1st and the 3rd, the 2nd and the 4th. It returns all recoupling coefficients for all outer multiplicities that occur in a matrix as follows,

Γ_1	Γ_2	Γ_{12}	ρ_{12}	1	...
Γ_3	Γ_4	Γ_{34}	ρ_{34}	1	...
Γ_{13}	Γ_{24}	Γ	$\rho_{13,24}$	1	...
ρ_{13}	ρ_{24}	$\rho_{12,34}$			
1	1	1		...	...
...	...	...		...	...

C. Calculation of reduced matrix elements of SU(3) generators

1. Reduced matrix elements of the angular momentum operator:

```
Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> RME_L(u3::physicalvector &bra,
            u3::physicalvector &ket, std::vector<int> &bra_kappa, std::vector<int> &ket_kappa)
```

Accepts two physical basis states as inputs and two empty arrays to store the multiplicity labels. It returns the reduced matrix elements

$$\langle(\lambda\mu)\kappa' L'||\mathbf{L}||(\lambda\mu)\kappa L\rangle = \sqrt{L(L+1)(2L+1)}$$

in form of a matrix whose number of rows is $\kappa'_{\max}$ and number of columns is $\kappa_{\max}$.

2. Reduced matrix elements of the quadrupole moment operator:

```
Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> RME_Q(u3::physicalvector &bra,
            u3::physicalvector &ket, std::vector<int> &bra_kappa, std::vector<int> &ket_kappa)
```

Accepts two physical basis states as inputs and two empty arrays to store the multiplicity labels. It returns the reduced matrix elements

$$\langle(\lambda\mu)\kappa' L'||\mathbf{Q}||(\lambda\mu)\kappa L\rangle$$

as given in Eqs. (51)-(53) of [3] with an extra factor of $\sqrt{2L'+1}$, in form of a matrix whose number of rows is $\kappa'_{\max}$ and number of columns is $\kappa_{\max}$. Note that with the extra factor in place, the conjugation relation becomes

$$\langle(\lambda\mu)\kappa L||\mathbf{Q}||(\lambda\mu)\kappa' L'\rangle = (-)^{L-L'}\langle(\lambda\mu)\kappa' L'||\mathbf{Q}||(\lambda\mu)\kappa L\rangle.$$

D. Utility functions

In directory `src/utils`, there are a number of utility functions that are used to find the inverse or the nullspace of a matrix. These uitlity functions are built based on the Eigen3 library, with separate handling of dense and sparse matrices.

1. Sparsity of a matrix, which is the ratio between the number of zero elements and the total number of element:

```
float sparisity(Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> &mat)
```

Returns a value between 0 and 1. Throughout the code, a matrix is considered to be sparse if its sparsity is greater than 0.7.

2. Solve an upper triangular matrix using backward substitution:

```
Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> backwardSubstitution(Eigen::Matrix<double,
                                        Eigen::Dynamic,Eigen::Dynamic> &mat, int nullity)
```

3. Perform orthonomalization on columns of a matrix using the modified Gram-Schmidt procedure:

```
Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic> gramSchmidt(Eigen::Matrix<double,
                                        Eigen::Dynamic,Eigen::Dynamic> &mat)
```

4. Find the inverse of a matrix:

```
Eigen::Matrix<double, Eigen::Dynamic,Eigen::Dynamic> inverse(Eigen::Matrix<double,
                                        Eigen::Dynamic,Eigen::Dynamic> &mat)
```

Wherein if the provided matrix is dense, it directly uses the inverse solver of Eigen3, otherwise – i.e., the matrix is sparse – it performs a QR decomposition and solves the equation $\mathbf{AX} = \mathbf{1}$.

5. Find the nullspace of a matrix:

```
Eigen::Matrix<double, Eigen::Dynamic,Eigen::Dynamic> null_space(Eigen::Matrix<double,
                                                Eigen::Dynamic,Eigen::Dynamic> &mat)
```

Returns 1 if the given matrix has one (or no) elememt. Otherwise, it performs a QR decomposition on the matrix (either a dense or a sparse solver depending on the sparsity of the matrix). After QR decomposition is performed, function `backwardSubstitution` is carried out for the `R` matrix, to which the solution must be orthonormalized using a `gramSchmidt` procedure. The final step is to adopt a simple and uniquely defined phase convention. This is what needs to be optimized, or perhaps we need to develop a new nullspace solver.

III. INSTALLATION

The U3lib library only contains header files, and users can compile .cpp files separately. Nonetheless, due to the utilization of two external libraries, Eigen3 and and WIGXJPF (which also contains only header files); these will need to be downloaded, installed and linked properly during compilation in a terminal command line.

[1] P. Dang, J. P. Draayer, F. Pan, and K. S. Becker, The European Physical Journal Plus **139**, 704 (2024).
[2] J. P. Draayer and G. Rosensteel, Nuclear Physics A **439**, 61 (1985).
[3] F. Pan, S. Yuan, K. D. Launey, and J. P. Draayer, Nuclear Physics A **952**, 70 (2016).